

GitHub Actions workflow yml file

```
name: Run API Tests

on:
  push:
    branches: [main]
  workflow_dispatch:
  schedule:
    - cron: '30 2 * * *' # 8:00 AM IST
    - cron: '30 13 * * *' # 7:00 PM IST

jobs:
  run-tests:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v3

      - name: Setup Node
        uses: actions/setup-node@v3
        with:
          node-version: '18'

      - name: Install Newman
        run: |
          npm install -g newman
          npm install -g newman-reporter-htmlextra

      - name: Run Collection
        run: |
          newman run 'Inwarranty-flow Collection.postman_collection.json' \
            -e QA.postman_environment.json \
            -r cli,htmlextra \
            -d testdata.csv

      - name: Upload Report
        if: always()
        uses: actions/upload-artifact@v4.6.2
        with:
          name: newman-html-report
          path: newman/*.html
```

Detailed Explanation

```
name: Run API Tests
```

The **name of the workflow** — it will show up in the GitHub Actions UI.

```
on:
  push:
    branches: [main]
  workflow_dispatch:
  schedule:
    - cron: '30 2 * * *' # 8:00 AM IST
    - cron: '30 13 * * *' # 7:00 PM IST
```

Trigger	What it does
push	Triggers the workflow when someone pushes code to the main branch.
workflow_dispatch	Allows manual run via the "Run Workflow" button in GitHub UI.
schedule	Runs automatically based on the provided cron timings (in UTC). These times are 8:00 AM and 7:00 PM IST.

```
jobs:
  run-tests:
    runs-on: ubuntu-latest
```

- `jobs:` – Top-level keyword where all jobs are defined.
- `run-tests:` – The **name of the job**.
- `runs-on: ubuntu-latest` – This job will run on GitHub's **Ubuntu VM** (Linux environment).

Step 1: Checkout Code

Pulls your code from the repository into the runner so the tests can access it.

```
- name: Checkout Code
  uses: actions/checkout@v3
```

2: Setup Node.js

```
- name: Setup Node
  uses: actions/setup-node@v3
  with:
    node-version: '18'
```

Installs Node.js v18 — needed to run Newman (which is a Node-based CLI).

Step 3: Install Newman & Reporter

```
- name: Install Newman
  run: |
    npm install -g newman
    npm install -g newman-reporter-htmlextra
```

Installs **Newman CLI** and the **HTML Extra Reporter** globally on the VM.

Step 4: Run Postman Collection

```
- name: Run Collection
  run: |
    newman run 'Inwarranty-flow Collection.postman_collection.json' \
      -e QA.postman_environment.json \
      -r cli,htmlextra \
      -d testdata.csv
```

- `newman run ...` → Runs your Postman collection.
- `-e` → Uses an environment file (`QA.postman_environment.json`).
- `-r` → Uses CLI output + HTML Extra reporter.
- `-d` → Supplies test data via CSV (`testdata.csv`).
- Make sure the file names are **spelled exactly** — spaces in filenames can break it.

Step 5: Upload HTML Report

```
- name: Upload Report
  if: always()
  uses: actions/upload-artifact@v4.6.2
  with:
    name: newman-html-report
    path: newman/*.html
```

Key	Purpose
if: always()	Ensures that this step runs even if previous steps fail (important for debugging failed test runs).
actions/upload-artifact	Uploads files from the VM so you can download them from the Actions UI .
name:	The label for the artifact.
path:	Looks for all .html files inside the newman/ folder. Make sure the report was exported to this folder in Step 4.